

1 What is `res.write()`?

`res.write()` is a **method in Node.js** used to **send a response to the client in chunks** when you're working with HTTP servers. It comes from the **`http.ServerResponse` object**, which is usually available as `res` in the callback of `http.createServer()`.

2 What does it mean?

- `res` = the response object that Node.js provides.
- `.write()` = a method that **writes data to the response body** that will eventually be sent to the client (browser, Postman, etc.).

So essentially:

It “writes” data to the outgoing HTTP response, but it **does not end the response**. You can call it multiple times to send multiple chunks of data.

3 Role of `res.write()`

- To **send partial data** to the client before finishing the response.
- Useful for streaming large files or sending data in parts (chunks).
- Must be followed by `res.end()` to **finish the response** and let the client know all data has been sent.

Example:

<code>const http = require('http');</code>
<code>http.createServer((req, res) => {</code>
<code> res.write("Hello "); // first chunk</code>
<code> res.write("World!"); // second chunk</code>
<code> res.end(); // finishes the response</code>
<code>}).listen(3000, () => console.log('Server running on 3000'));</code>
Output in browser:
Hello World!

4 Where it comes from?

- `res.write()` comes from **Node.js's built-in `http` module**, specifically from the **`http.ServerResponse` class**.
- When you do `http.createServer((req, res) => {...})`, Node.js automatically gives you a `res` object, which has `.write()`, `.end()`, `.setHeader()`, etc.

 **Key point:**

- `res.write()` → writes data **but does not finish** the response.
- `res.end()` → finishes the response, optionally writing the last piece of data.